

实践9：BeyondMimic关键函数实现与轨迹追踪训练



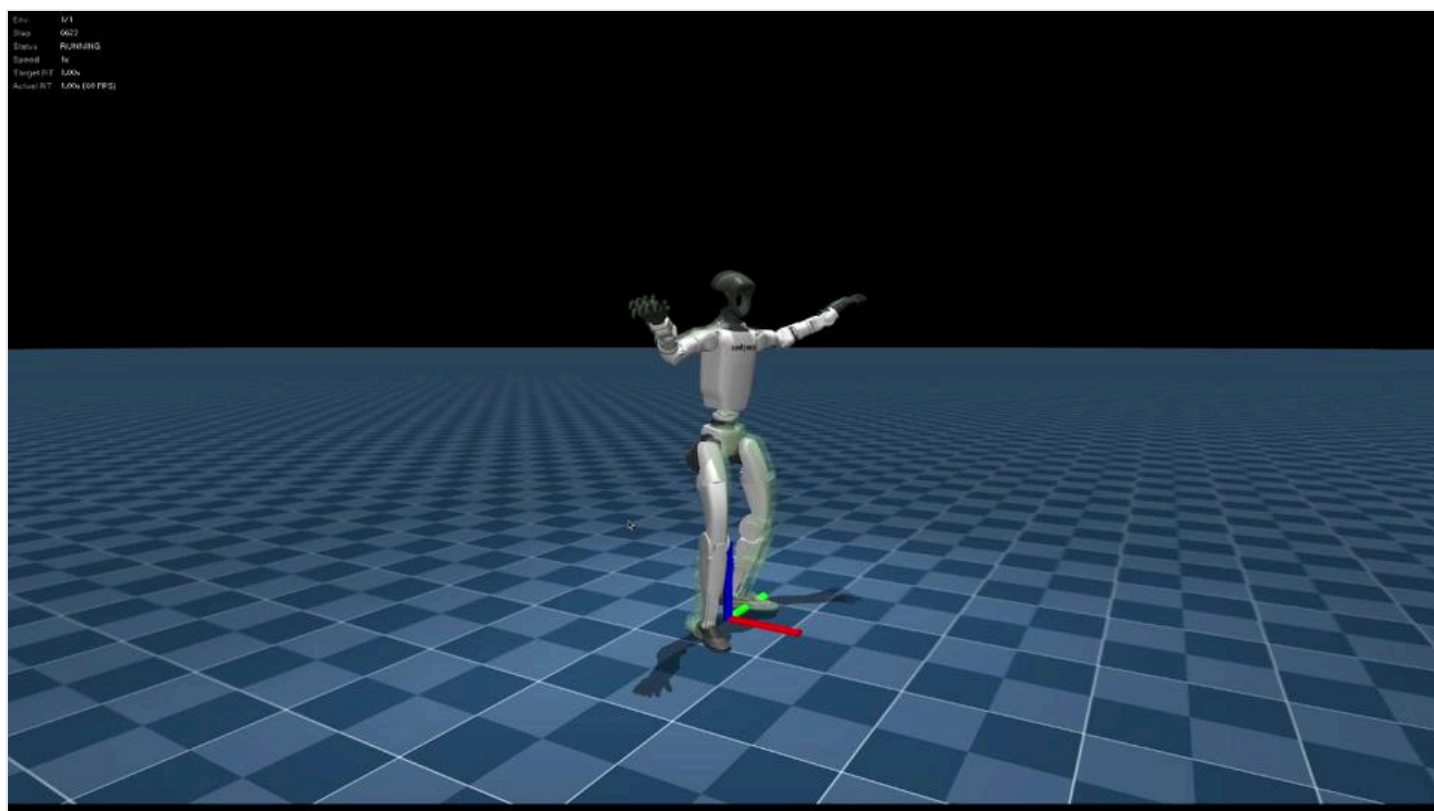
深蓝学院《人形机器人运动控制》课程第6章

课程链接：<https://www.shenlanxueyuan.com/course/864>

1. 作业背景

在机器人运动控制任务中，策略网络通常需要根据一段预先给定的参考动作，控制机器人在仿真环境中复现对应的身体姿态和运动轨迹。

本次实践使用 IsaacLab 或者 MJLAB 搭建单一轨迹跟踪任务。需要完成轨迹命令生成模块中的两个核心函数，并在此基础上完成一次完整的强化学习训练、指标记录和结果可视化。



本作业重点**不在于**修改网络结构或重新设计强化学习算法，**而在于**理解以下流程：

1. 参考动作如何转换为机器人当前时刻的跟踪命令；
2. 训练过程中如何根据各时间段的跟踪误差，自适应地调整动作片段的采样概率；

3. 完成代码实现后，如何运行测试、启动训练、查看训练曲线并导出结果视频。

 本次实践的代码量较少，但涉及轨迹索引、状态更新、概率采样和仿真器接口调用。使用 AI 辅助完成代码时，**应确保能够解释代码的实现逻辑、张量维度和修改原因。**

 **风险提示：**

自行训练得到的模型请勿直接部署到真机运行，操作不当极易造成机器人硬件损坏。

2. 作业任务

本次作业分为两个部分。

作业一：核心函数实现

在轨迹命令模块 `commands.py` 中完成以下两个函数：

- `_adaptive_sampling`
- `_update_command`

完成后，使用提供的测试脚本验证：

- 仿真器相关接口是否保持完整；
- 自适应采样逻辑是否正确；
- 轨迹命令更新逻辑是否正确。

作业二：单一轨迹跟踪训练

使用提供的动作文件 `dance1_subject2.csv`：

1. 生成与配置动作文件路径；
2. 启动 Unitree G1 单一轨迹跟踪训练；
3. 使用 TensorBoard 或 Weights & Biases 记录训练指标；
4. 导出训练结果可视化视频；
5. 整理代码、测试截图、训练曲线和视频并打包提交。

3. 前置准备

3.1 环境要求

通用环境要求

- 操作系统：Ubuntu 22.04

- 显卡：NVIDIA RTX 系列（显存 $\geq$ 8GB）
- 显卡驱动：要求CUDA $\geq$ 12.1
- Python：3.11

如果选用 MJLAB，建议采用以下版本

- MJLAB 1.2.0
- MuJoCo 3.7.0

如果选用 IsaacLab，建议采用以下版本

- Isaac Sim 5.0.0 或更新版本
- IsaacLab 2.2.0 或更新版本

注意：若在之前的实践中，使用了Unitree-RL系列仓库，则本项目可以复用前面实践的仓库，请确认版本号是否与上述期望一致。

3.2 选择实现路线

首先选择以下其中一条路线：

- 路线 A：MJLAB，默认使用 [Unitree_RL-mjlab](#) 仓库作为基仓库
- 路线 B：IsaacLab，默认使用 [Unitree_RL-Lab](#) 仓库作为基仓库

不建议在同一个 Python 环境中同时混装两套仿真框架。最好为 MJLAB 和 IsaacLab 分别创建独立环境。

3.3 备份原始文件

提供的 TODO 文件将替换仓库原有的 `commands.py`。原始文件中可能包含完整实现或相似实现，因此替换前必须做好备份。例如：

代码块

```
1 cp commands.py commands_backup.py
```

备份文件仅用于环境恢复和接口对照。学员提交的代码应为自己完成的 TODO 版本。

4. 作业环境准备

注意：下列替换的 `commands.py` 原始文件内包含部分作业答案或答案的变种，如有需要，请做好备份。我们会提供与作业符合的答案和与上述仓库版本对应的备份，不保证与你的`commands.py`一致。

4.1 如果选用MJLAB

默认使用 Unitree_RL-mjlab 仓库作为基仓库，仓库链接：

https://github.com/unitreerobotics/unitree_rl_mjlab

作业文件

- `bymic_commands_mjlab_todo.py`
- `bymic_commands_mjlab_test.py`

目标目录

代码块

```
1 unitree_rl_mjlab/src/tasks/tracking/mdp
```

文件替换步骤

进入目标目录，将原文件备份：

代码块

```
1 cd unitree_rl_mjlab/src/tasks/tracking/mdp
2 cp commands.py commands_backup.py
```

将提供的 TODO 文件复制到该目录，并重命名为 `commands.py`：

代码块

```
1 cp /path/to/bymic_commands_mjlab_todo.py ./commands.py
```

其中 `/path/to/` 需要替换为 TODO 文件的实际路径。

4.2 如果选用IsaacLab

默认使用 Unitree_RL-Lab 仓库作为基仓库，仓库链接：

https://github.com/unitreerobotics/unitree_rl_lab

作业文件

- `bymic_commands_isaacLab_todo.py`
- `bymic_commands_isaacLab_test.py`

目标目录

代码块

```
1 unitree_rl_lab/source/unitree_rl_lab/unitree_rl_lab/tasks/mimic/mdp
```

文件替换步骤

进入目标目录，将原文件备份：

代码块

```
1 cd unitree_rl_lab/source/unitree_rl_lab/unitree_rl_lab/tasks/mimic/mdp
2 cp commands.py commands_backup.py
```

将提供的 TODO 文件复制到该目录，并重命名为 `commands.py`：

代码块

```
1 cp /path/to/bymic_commands_isaacLab_todo.py ./commands.py
```

注意：IsaacLab 路线应使用 `bymic_commands_isaacLab_todo.py`，不要误用 MJLAB 版本。

5. 作业一：核心函数实现

请在 `bymic_commands_[sim]_todo.py` 中完成以下两个函数，并保持原有函数签名和类接口不变。

TODO 1: 实现 `_adaptive_sampling`

根据各动作时间区间的历史失败情况，自适应调整采样概率，并为需要重置的环境选择新的动作起始帧。

主要任务：

1. 获取重置环境的失败状态；
2. 将当前动作帧映射到对应的时间 bin；
3. 统计各 bin 的失败次数；
4. 根据历史失败次数构造并平滑采样概率；
5. 对概率进行归一化并采样新的动作帧；
6. 更新采样熵、最高概率和最高概率 bin 等指标。

要求：

- 采样概率非负且总和为 1；
- 动作帧索引不得越界；
- 使用 PyTorch 张量运算；
- 保持张量设备和数据类型一致。

TODO 2: 实现 `_update_command`

推进参考动作时间，并将参考动作按照机器人当前锚点的位置和朝向进行对齐。

主要任务：

1. 将动作时间推进一帧；
2. 对播放结束的环境重新采样；
3. 计算参考锚点与机器人锚点之间的位置和朝向差；
4. 根据锚点变换更新目标刚体的位置和姿态；
5. 使用指数滑动平均更新历史失败统计；
6. 清空本轮临时失败计数。

要求：

- 姿态对齐只保留 yaw 方向；
- 不得修改原始动作数据；
- 输出张量形状、设备和四元数格式必须与原接口一致。

具体成员变量和辅助函数可参考 TODO 文件中的注释。MJLAB 与 IsaacLab 的接口有所不同，请使用对应版本实现。

6. 作业一测试

MJLAB 版本

代码块

```
1 python bymic_commands_mjlab_test.py /path/to/your/bymic_commands_mjlab_todo.py
```

IsaacLab 版本

代码块

```
1 python bymic_commands_isaacLab_test.py  
  /path/to/your/bymic_commands_isaacLab_todo.py
```

若完成所有测试且逻辑符合预期，则输出：

代码块

```
1 (env) shenlan@shenlan:~/path/to/project$ python bymic_commands_mjlab_test.py  
  ./bymic_commands_mjlab_todo.py
```

```
2 Test 0 passed: MJLab interfaces are intact.
3 Test 1 passed: _adaptive_sampling matches the reference implementation.
4 Test 2 passed: _update_command matches the reference implementation.
5
6 All tests passed for /path/to/your/bymic_commands_mjlab_todo.py.
```

若报错，则按报错修改即可。

7. 作业二：完成单一轨迹跟踪训练

作业一测试通过后，再开始作业二的训练。作业二需完成完整的训练流程与结果展示。

先下载提供的动作原始文件：

代码块

```
1 dance1_subject2.csv
```

MJLAB 版本

在项目根目录下运行：

代码块

```
1 python scripts/csv_to_npz.py \
2 --input-file path/to/dance1_subject2.csv \
3 --output-name path/to/your/target/path/dance1_subject2.npz \
4 --input-fps 30 \
5 --output-fps 50 \
6 --robot g1
```

记录参数 `--output_name` 所指向的路径。

定位到 `src/tasks/tracking/config/g1/env_cfgs.py` 文件，在第 41 行的 `assert isinstance(motion_cmd, MotionCommandCfg)` 的语句之后，新增如下一行代码，用于指定训练所用的动作文件，该路径指向上一命令生成的 npz 文件：

代码块

```
1 motion_cmd.motion_file = "path/to/our/dance1_subject2.npz"
```

然后，使用以下指令启动训练。

代码块

```
1 python scripts/train.py Unitree-G1-Tracking --env.scene.num-envs=2048
```



环境数可根据自己计算机的情况调整，参数设置建议见后文。

IsaacLab 版本

在项目根目录下执行

代码块

```
1 python scripts/mimic/csv_to_npz.py -f path/to/dance1_subject2.csv --input_fps  
60
```

默认生成到与csv同路径下的同名.npz文件，记录该文件路径。

定位到

`source/unitree_rl_lab/unitree_rl_lab/tasks/mimic/robots/g1_29dof/dance_102/tracking_env_cfg.py` 文件，在第 91 行的 `motion_file=f"{os.path.dirname(__file__)} /G1_Take_102.bvh_60hz.npz"`，的语句之后，新增如下一行代码，用于指定训练所用的动作文件，该路径指向上一命令生成的npz文件：

代码块

```
1 motion_file="path/to/our/dance1_subject2.npz",
```

然后，使用以下指令启动训练。

代码块

```
1 python scripts/rsl_rl/train.py --task Unitree-G1-29dof-Mimic-Dance-102 --  
num_envs 2048
```



环境数可根据自己计算机的情况调整，参数设置建议见后文。

8. 训练参数设置建议

助教做测试时，使用的设备参数：

- CPU: Intel i9-14900K
- GPU: NVIDIA RTX 5090

- 内存: 64GB

设定4096环境，训练2w步可达到理想结果。

若降低并行环境数目，需要延长训练步数，参考计算公式：

$$Env_{your} \times Step_{your} = 4096 \times 20000$$

其中： Env 代表你的实际并行环境数； $Step$ 代表迭代回合数，也记作 $Iteration$ 。

9. 作业提交要求

1. 将完成两个函数后的 `commands.py` 复制并重命名为 `P1_commands_todo.py` 后提交。
2. 提交test测试的命令行输出截图，命名为 `P1_commands_test_output.png`。
PNG、JPG、JPEG是受支持的图片格式，其他视为无效提交。
3. 提交训练曲线，只接受 wandb 或 tensorboard，需要截取下列曲线，合并为一张图或分别截取多张，命名为 `P2_scalars_[name_of_metric].png` 或 `P2_scalars_all.png`。
PNG、JPG、JPEG是受支持的图片格式，其他视为无效提交。
 - a. Train/mean_episode_length
 - b. Episode_Reward/motion_body_pos
 - c. Episode_Termination/time_out
 - d. Metrics/motion/sampling_entropy
 - e. Metrics/motion/sampling_top1_bin
 - f. Metrics/motion/error_joint_pos
 - g. Policy/mean_std
4. 提交训练结果可视化视频，长度不超过30s，命名为 `P2_play.mp4`。MP4和MOV是受支持的视频格式，其他视为无效提交。
5. 请将上述材料打包为zip文件提交，命名为 `用户名-实践9.zip`。

除zip以外的格式是不受支持的格式，视为无效提交。

10. 评分参考

项目	分值	检查重点
算法实现与接口对齐	50%	失败统计、概率采样、仿真器数据正确获取
测试通过	10%	test 正常输出
曲线符合预期	20%	机器人有效学习，采样分布合理

真实训练

20%

机器人正确跟踪轨迹